

DevOps Interview Preparation Guide

Prepared for: Sanket Ajay Chopade	Role: DevOps / Cloud Engineer
University: RTM Nagpur University	Experience: Intern – Hisan Labs

Comprehensive structured answers across 16 interview categories — HR, Technical, Project-Based, Scenario, and Behavioural.

1. HR & Introduction Questions

Q. Tell me about yourself.

My name is Sanket Ajay Chopade. I recently completed my B.Tech from Rashtrasant Tukadoji Maharaj Nagpur University. During my internship at Hisan Labs, I gained hands-on experience in cloud and DevOps technologies.

I worked with AWS, Terraform, Docker, Kubernetes, Jenkins, GitHub Actions, and ArgoCD. Key projects include a Three-Tier DevSecOps deployment on AWS EKS and a Cloud-Native GitOps Delivery Pipeline.

I am passionate about automation, cloud infrastructure, and continuous improvement. My goal is to grow into a skilled DevOps or Cloud Engineer, contributing to reliable and scalable systems.

Q. Why did you choose DevOps as a career?

I was drawn to DevOps because it bridges development and operations — two areas I find equally exciting. I enjoy automation, solving infrastructure problems, and building pipelines that accelerate software delivery.

During college, I discovered that cloud-native tools like Kubernetes and CI/CD pipelines were revolutionising how software is built and deployed, and I wanted to be part of that shift.

■ *Mention interest in automation, reliability, and modern software delivery practices.*

Q. What are your strengths?

- Strong problem-solving mindset — I debug issues systematically using logs, metrics, and describe commands.
- Quick learner — I independently learned Terraform, Kubernetes, and ArgoCD through hands-on projects.
- Automation-first thinking — I always look for ways to reduce manual work through scripting and pipelines.
- Attention to security — I integrated SonarQube and Trivy into CI/CD pipelines to catch issues early.

Q. What is your biggest weakness?

Early in my internship, I occasionally over-engineered solutions before fully understanding requirements. I have since learned to first clarify the problem scope, prototype quickly, and iterate based on feedback.

■ *Always pair a weakness with the corrective action you have taken.*

Q. Where do you see yourself in 5 years?

In five years, I aim to be a senior DevOps or Platform Engineer responsible for designing and maintaining large-scale cloud infrastructure. I also want to deepen expertise in Site Reliability Engineering (SRE) and contribute to open-source DevOps tooling.

Q. Why should we hire you?

- Practical, project-backed experience with AWS, Kubernetes, Terraform, Jenkins, and ArgoCD.
- Demonstrated ability to build end-to-end secure CI/CD pipelines with DevSecOps tooling.
- Strong learning agility — comfortable picking up new tools independently.
- Career-focused mindset aligned with modern cloud-native practices.

2. Internship Experience – Hisan Labs

Q. What were your daily responsibilities?

- Writing and maintaining Terraform modules for AWS infrastructure provisioning.
 - Managing Kubernetes deployments, services, and config maps on EKS.
 - Building and troubleshooting Jenkins CI/CD pipelines.
 - Monitoring application health using Prometheus, Grafana, Datadog, and CloudWatch.
 - Collaborating with developers to support deployments and resolve environment issues.
-

Q. What was your biggest contribution during the internship?

My most significant contribution was designing and implementing a Three-Tier DevSecOps pipeline on AWS EKS. This included automated infrastructure provisioning with Terraform, secure CI/CD via Jenkins with SonarQube and Trivy scanning, and GitOps-based deployments using ArgoCD — reducing manual deployment effort and improving security posture.

Q. What challenges did you face and how did you overcome them?

- Terraform state conflicts — resolved by enabling S3 remote state with DynamoDB state locking.
 - Pods entering CrashLoopBackOff — debugged using `kubectl logs` and `kubectl describe pod`; fixed resource limits and misconfigured env variables.
 - Jenkins pipeline credential issues — fixed by using Jenkins credentials store and referencing secrets securely in the pipeline.
 - ArgoCD OutOfSync issues — resolved by checking manifest drift and ensuring the Git source was the single source of truth.
-

3. Project Deep Dive

■ Three-Tier DevSecOps on AWS EKS

Q. Explain the project architecture.

The project follows a three-tier architecture:

- Presentation Tier: React/Nginx frontend served via AWS Application Load Balancer (ALB).
- Application Tier: Backend API (Node.js / Python) deployed as Kubernetes pods on AWS EKS.
- Data Tier: Amazon RDS (PostgreSQL) in a private subnet, accessible only by the application tier.

Traffic flows: Internet → ALB → Kubernetes Ingress → Application pods → RDS. Infrastructure is provisioned entirely via Terraform.

Q. Explain the end-to-end CI/CD pipeline.

- Developer pushes code to GitHub.
- Jenkins picks up the change via a webhook trigger.
- Pipeline stages: Code Checkout → SonarQube static analysis → Unit Tests → Trivy image vulnerability scan → Docker Build → Push to Amazon ECR.
- ArgoCD detects the updated image tag in the Git manifest repository and auto-syncs to EKS.
- Kubernetes rolling update deploys the new version with zero downtime.

Q. Why EKS over ECS or plain EC2?

EKS was chosen because Kubernetes provides a richer ecosystem — native support for Helm, ArgoCD, GitOps, HPA autoscaling, and a large community.

ECS is simpler but less portable. EC2 requires managing the orchestration layer manually. EKS lets us use standard Kubernetes tooling while AWS handles the control plane.

Q. Why Terraform over CloudFormation?

- Multi-cloud portability — Terraform is not AWS-specific.
- HCL is more readable and modular than JSON/YAML CloudFormation templates.
- Terraform modules allow reuse across environments (dev, staging, prod).
- Stronger community ecosystem and provider support.

Q. Why Jenkins over GitHub Actions?

Jenkins was chosen for its self-hosted, highly customisable pipeline capabilities, advanced plugin ecosystem, and the ability to run on private infrastructure — critical for security-sensitive pipelines that need to access private ECR, SonarQube, and internal registries without exposing tokens to a SaaS platform.

Q. Why SonarQube and Trivy?

- SonarQube: Performs SAST (Static Application Security Testing) — detects code smells, bugs, and vulnerabilities in source code before the build stage.
- Trivy: Scans Docker images for OS-level and application-level CVEs (Common Vulnerabilities and Exposures) before pushing to ECR, ensuring only clean images are deployed.

■ Cloud-Native GitOps Delivery Pipeline (Go App)

Q. What is GitOps and how did you implement it?

GitOps is a deployment methodology where Git is the single source of truth for both application code and infrastructure configuration. Any change to the Git repository automatically triggers a reconciliation loop that brings the cluster state in sync.

Implementation: GitHub Actions built and pushed the Docker image on every push to main. ArgoCD watched the manifest repository; when the image tag was updated via a GitHub Actions step, ArgoCD automatically synced the new deployment to the Kubernetes cluster.

Q. How does ArgoCD detect and apply changes?

- ArgoCD continuously polls the Git repository (default: every 3 minutes) or listens to webhooks.
 - It compares the desired state in Git against the live state in the Kubernetes cluster.
 - If drift is detected, ArgoCD marks the app as OutOfSync and can auto-sync (if configured) or wait for manual approval.
 - Changes are applied using `kubectl apply` under the hood, respecting Kubernetes rollout strategies.
-

4. AWS Core Concepts

Q. What is a VPC and how did you structure it for EKS?

A Virtual Private Cloud (VPC) is an isolated network environment in AWS. For EKS, the VPC was structured as:

- Public subnets (2 AZs): Hosting the ALB and NAT Gateways.
- Private subnets (2 AZs): Hosting EKS worker nodes and RDS — not directly reachable from the internet.
- Internet Gateway: Attached to the VPC for public subnet internet access.
- NAT Gateway: Allows private subnet resources (worker nodes) to reach the internet for pulling images without being publicly exposed.

Q. Explain IAM roles in EKS.

- EKS Cluster IAM Role: Grants the control plane permission to manage AWS resources on your behalf (EC2, ELB, etc.).
- Node Group IAM Role: Grants worker nodes permissions to pull from ECR, write CloudWatch logs, and interact with EKS.
- IRSA (IAM Roles for Service Accounts): Allows individual Kubernetes pods to assume specific IAM roles — enabling least-privilege access. For example, only the pod that needs S3 access gets an IAM role for S3.

■ *IRSA is preferred over attaching broad permissions to the node IAM role.*

Q. Types of AWS Load Balancers and when to use each?

- Application Load Balancer (ALB): Layer 7 (HTTP/HTTPS), path-based and host-based routing. Used for web applications — chosen for this project.
- Network Load Balancer (NLB): Layer 4 (TCP/UDP), ultra-low latency, static IPs. Used for non-HTTP traffic or extreme performance needs.
- Gateway Load Balancer (GWLB): For third-party virtual appliances like firewalls.

Q. Scenario: One EKS node becomes NotReady — what do you do?

- Run: `kubectl describe node` to check conditions and events.
- SSH into the node (if accessible) and check: `systemctl status kubelet`.
- Check if the node has disk pressure, memory pressure, or PID pressure via `kubectl get nodes`.
- Review CloudWatch agent logs and node system logs.
- If unrecoverable, drain and cordon the node: `kubectl drain --ignore-daemonsets`, then terminate and let the Auto Scaling Group replace it.

■ *AWS Managed Node Groups auto-replace failed nodes via ASG health checks.*

5. Terraform – Infrastructure as Code

Q. What is Terraform State and why is it important?

Terraform state is a JSON file (terraform.tfstate) that maps your configuration to real-world resources. Terraform uses it to determine what changes are needed on each plan/apply.

- Without state, Terraform cannot know what it has already created.
- Remote state (S3 + DynamoDB) was used in the project for team collaboration and state locking.
- State locking prevents two engineers from running terraform apply simultaneously, avoiding corruption.

Q. Scenario: Terraform apply failed halfway — how do you recover?

- Run terraform plan to see the current state vs desired state.
- Identify which resources were partially created using the AWS Console or CLI.
- If safe, re-run terraform apply — Terraform is idempotent and will only create/modify what is missing.
- For resources that are in a broken state, use terraform taint to mark for recreation, then apply again.
- If the state file is inconsistent, use terraform state rm to remove orphaned entries and import existing resources with terraform import.

Q. What are Terraform Modules and why use them?

Modules are reusable, self-contained Terraform configurations. In the EKS project, separate modules were created for: VPC, EKS Cluster, Node Groups, and RDS.

- Reusability: The same VPC module is used for dev and prod with different variable values.
 - Maintainability: Isolates changes — modifying the EKS module does not affect the VPC module.
 - Consistency: Enforces standard patterns across environments.
-

6. Docker – Containerisation

Q. Difference between a Docker image and a Docker container?

- Image: A read-only, immutable snapshot of the filesystem and configuration (built from a Dockerfile). It is a blueprint.
 - Container: A running instance of an image. Containers are ephemeral and isolated processes with their own filesystem, network, and process namespace.
-

Q. Key Dockerfile instructions — what do they do?

- FROM: Sets the base image (e.g., FROM node:18-alpine).
 - RUN: Executes commands during the image build (e.g., installing packages).
 - COPY / ADD: Copies files from host into the image. ADD additionally supports URL downloads and auto-extraction of archives.
 - CMD: Default command run when the container starts (can be overridden at runtime).
 - ENTRYPOINT: Fixed command that always runs; CMD provides its default arguments.
 - WORKDIR: Sets the working directory for subsequent instructions.
 - EXPOSE: Documents the port the container listens on (does not publish it).
-

Q. Scenario: Docker image size is too large — how do you reduce it?

- Use a minimal base image: alpine or distroless instead of ubuntu.
 - Use multi-stage builds: compile in a build stage, copy only the binary into a clean runtime stage.
 - Combine RUN commands using && to reduce layer count.
 - Remove build tools and caches in the same RUN step: apt-get clean && rm -rf /var/lib/apt/lists/*.
 - Use .dockerignore to exclude node_modules, .git, and test files from the build context.
-

7. Kubernetes (K8s) – Container Orchestration

Q. Explain Kubernetes architecture.

- Control Plane: API Server (central gateway), etcd (key-value store for cluster state), Scheduler (assigns pods to nodes), Controller Manager (manages ReplicaSets, deployments, etc.).
- Worker Nodes: kubelet (node agent, ensures pods are running), kube-proxy (maintains network rules), Container Runtime (containerd / CRI-O).
- In AWS EKS, AWS manages the control plane; you manage the worker nodes via node groups.

Q. Difference between Deployment, ReplicaSet, and Pod?

- Pod: Smallest deployable unit — one or more containers sharing network and storage.
- ReplicaSet: Ensures a specified number of identical pods are always running.
- Deployment: A higher-level abstraction that manages ReplicaSets, enabling rolling updates and rollbacks.

Q. Difference between ClusterIP, NodePort, LoadBalancer, and Ingress?

- ClusterIP: Internal service — accessible only within the cluster. Used for backend-to-backend communication.
- NodePort: Exposes the service on a static port on each node's IP. Not production-grade.
- LoadBalancer: Provisions a cloud load balancer (ALB/NLB) to expose the service externally.
- Ingress: Layer 7 routing — routes external HTTP/HTTPS traffic to multiple services based on host/path rules. Requires an Ingress Controller (e.g., AWS Load Balancer Controller).

Q. Scenario: Pod is in CrashLoopBackOff — how do you troubleshoot?

- `kubectl logs --previous` — view logs from the previous (crashed) container.
- `kubectl describe pod` — check Events for OOMKilled, image pull errors, liveness probe failures.
- Common causes: application crashing on startup, misconfigured env variables, missing ConfigMap/Secret, insufficient memory limits (OOMKilled), liveness probe misconfigured.
- Fix: Correct the misconfiguration, update the manifest in Git, and let ArgoCD redeploy.

■ *Always check both logs and describe events — they give complementary information.*

Q. What is HPA (Horizontal Pod Autoscaler)?

HPA automatically scales the number of pod replicas based on observed metrics such as CPU or memory utilisation (or custom metrics via the metrics server).

- Example: If CPU exceeds 70%, HPA scales the deployment from 2 pods to 5 pods.
- Works in conjunction with Cluster Autoscaler, which adds new worker nodes if pods cannot be scheduled due to insufficient node capacity.

8. Jenkins – CI/CD Pipelines

Q. Explain your Jenkins pipeline stages.

- Checkout: Pulls source code from the GitHub repository.
- Code Quality: SonarQube performs static analysis. Quality Gate must pass to proceed.
- Unit Tests: Runs application test suite; pipeline fails if tests fail.
- Build: Compiles the application / installs dependencies.
- Docker Build: Builds the Docker image using the Dockerfile.
- Security Scan: Trivy scans the Docker image for CVEs.
- Push: Authenticated push of the image to Amazon ECR.
- Update Manifest: Updates the Kubernetes deployment manifest in the GitOps repo with the new image tag.
- ArgoCD Sync: ArgoCD detects the manifest change and deploys to EKS.

Q. Declarative vs Scripted Pipeline — what did you use and why?

Declarative Pipeline was used because it provides a structured, readable syntax with built-in validation, making it easier to maintain and review. Scripted pipeline uses Groovy and is more flexible but harder to read and audit.

■ *Declarative is the industry standard for most modern Jenkins pipelines.*

Q. Scenario: Jenkins pipeline fails at the Docker build stage. What do you check?

- Review the build logs in Jenkins for the exact error message.
 - Check if the Dockerfile has syntax errors using docker build locally.
 - Verify the ECR login credentials (aws ecr get-login-password) in Jenkins credentials store.
 - Ensure the Jenkins agent has Docker installed and the Docker daemon is running.
 - Check disk space on the Jenkins agent — a full disk causes Docker build failures.
-

9. ArgoCD & GitOps

Q. What is the difference between GitOps and traditional CI/CD?

- Traditional CI/CD: The pipeline directly pushes changes to the cluster (push-based). If the cluster drifts, there is no automatic correction.
- GitOps (pull-based): The cluster pulls its desired state from Git. ArgoCD continuously reconciles actual state to desired state. Drift is automatically detected and corrected.
- Key benefit: Git becomes an immutable audit log. Any change to the cluster must go through a Git commit — providing traceability, rollback capability, and access control via pull requests.

Q. Scenario: ArgoCD shows the application as OutOfSync — what do you do?

- Open the ArgoCD UI and check the Diff view to see what has changed between Git and the live cluster.
- Identify whether the drift was intentional (a hotfix applied directly to the cluster) or accidental.
- If accidental: click Sync in ArgoCD (or run `argocd app sync`) to restore the Git-desired state.
- If intentional: commit the change back to Git so it becomes the new desired state.

■ *Never make direct `kubectl` apply changes to production — always go through Git.*

10. Monitoring & Observability

Q. How does Prometheus collect metrics?

Prometheus uses a pull model — it scrapes HTTP `/metrics` endpoints exposed by targets at regular intervals (default: 15 seconds). Exporters (e.g., `node_exporter` for Linux metrics, `kube-state-metrics` for Kubernetes) expose metrics in the Prometheus format.

- In the EKS project: Node Exporter → node CPU/memory metrics; kube-state-metrics → pod/deployment status; Application → custom business metrics.
- AlertManager was configured to send alerts to a notification channel when thresholds were breached.

Q. How does Grafana integrate with Prometheus?

Grafana is added as a data source pointing to the Prometheus server URL. Dashboards are built using PromQL queries. Pre-built dashboards (e.g., Node Exporter Full, Kubernetes cluster monitoring) were imported from grafana.com to visualise cluster health at a glance.

Q. Scenario: CPU suddenly reaches 95% on a pod — what do you do?

- Check Grafana dashboards for the exact pod/container causing the spike.
- Run `kubectl top pod --containers` to confirm live CPU usage.
- `kubectl logs` — check for infinite loops or runaway processes.
- `kubectl describe pod` — check if OOMKilled events or resource limits are too low.
- If a traffic spike: trigger HPA or manually scale: `kubectl scale deployment --replicas=5`.
- If a code bug: roll back the deployment: `kubectl rollout undo deployment/`.

11. DevSecOps – Security in the Pipeline

Q. What is 'shift-left security' and how did you apply it?

Shift-left security means integrating security checks early in the development lifecycle — during code commit and build — rather than at the end (production).

- SonarQube: Scans source code for vulnerabilities, bugs, and code smells during the CI pipeline, before the Docker image is built.
- Trivy: Scans the Docker image for OS and library CVEs before it is pushed to ECR.
- IAM least privilege: Each component (pods via IRSA, Terraform runner) has only the permissions it needs.

■ *Catching a vulnerability in the pipeline is 100x cheaper than patching it in production.*

Q. Scenario: Trivy detects a CRITICAL vulnerability in the Docker image — what do you do?

- The Jenkins pipeline is configured to fail the build if CRITICAL CVEs are found — blocking the deployment automatically.
- Identify the vulnerable package: `trivy image --severity CRITICAL`.
- Update the base image (FROM) to a patched version or upgrade the vulnerable dependency in the Dockerfile.
- Rebuild and re-scan. Only push to ECR once the image passes the Trivy scan.

■ *If an urgent hotfix is needed, temporarily pin to a known-clean image tag while the fix is applied.*

12. Linux & Shell Scripting

Q. Scenario: Disk is full on a server — what do you do?

- `df -h` — identify which filesystem is full.
- `du -sh /var/log/* | sort -rh | head -20` — find the largest directories/files.
- Clear old Docker images/containers: `docker system prune -af`.
- Rotate or archive large log files in `/var/log`.
- Set up log rotation (`logrotate`) and CloudWatch log streaming to prevent recurrence.

Q. Key Linux commands for a DevOps Engineer?

- `ps aux | grep` — list running processes.
- `top` / `htop` — real-time CPU and memory monitoring.
- `netstat -tlnp` / `ss -tlnp` — list open ports and listening services.
- `grep -r 'ERROR' /var/log/` — search logs for errors.
- `chmod` / `chown` — manage file permissions and ownership.
- `curl -I` — check HTTP headers and response status.
- `systemctl status` / `restart` — manage system services.

13. Behavioural & Situational Questions

Q. Tell me about a challenge you faced during your internship.

During the Three-Tier EKS project, ArgoCD was not syncing because the Kubernetes manifests had a misconfigured resource limit that caused pods to enter a Pending state. The cluster had no visible error at first glance.

I systematically ran `kubectl describe pod`, examined Events, and traced the issue to an incorrect memory request that exceeded node capacity. I updated the manifest, committed to Git, and ArgoCD successfully synced on the next reconciliation cycle.

■ Use the STAR method: Situation, Task, Action, Result.

Q. Describe a time you worked under pressure.

During a critical deployment, the Jenkins pipeline failed just before a deadline due to a stale ECR authentication token. I quickly diagnosed the issue using Jenkins logs, regenerated the token via `aws ecr get-login-password`, updated the credential in Jenkins, and re-triggered the pipeline. The deployment completed successfully within 20 minutes.

Q. How do you prioritise multiple tasks?

- Assess impact and urgency — production issues always take priority.
- Break large tasks into smaller deliverables with clear milestones.
- Communicate blockers early to the team or supervisor.
- Use tools like GitHub Issues or Jira to track and organise work.

14. Real-Time Production Scenario Questions

Q. Production website is down — what steps do you take?

- Step 1: Check ALB health in the AWS Console — is the target group reporting unhealthy instances?
- Step 2: `kubectl get pods -n` — are pods running?
- Step 3: `kubectl logs` — check for application errors.
- Step 4: Check Grafana / CloudWatch dashboards for CPU, memory, and error rate spikes.
- Step 5: If a recent deployment caused it — `kubectl rollout undo deployment/`
- Step 6: Notify stakeholders with status and ETA. Document the incident after resolution.

■ Always establish an incident timeline and do a post-mortem with root cause analysis.

Q. Load balancer returns 502 Bad Gateway errors — what is the cause?

- 502 means the ALB received an invalid response from the backend. The backend pods are likely down or returning errors.
- Check: `kubectl get pods` — are all pods Running?
- Check: `kubectl logs` — is the application throwing unhandled exceptions?
- Check: EKS target group health in the AWS Console — are targets healthy?
- Check: Resource limits — is the pod OOMKilled (memory exhausted)?
- Fix: Scale up pods, fix the application error, or roll back the last deployment.

Q. SSL certificate expires — how do you handle it?

- Immediate: If using AWS Certificate Manager (ACM), certificates are auto-renewed. Check if auto-renewal is configured for the domain.

- If self-managed (cert-manager on K8s): Check certificate resource: `kubectl describe certificate -n .`
 - If cert-manager failed to renew: `kubectl delete certificate` and let cert-manager re-issue.
 - Prevention: Set up CloudWatch Alarms to alert 30 days before expiry.
-

15. Most Important — Almost Guaranteed Questions

Q. Difference between Docker and Kubernetes?

- Docker: A container runtime that packages applications into portable, isolated containers.
- Kubernetes: A container orchestration platform that automates deployment, scaling, load balancing, self-healing, and rolling updates of containers across a cluster of nodes.

Analogy: Docker is the shipping container; Kubernetes is the shipping port that manages thousands of containers across multiple ships (nodes).

Q. Explain Terraform state — what happens if it is deleted?

If the state file is deleted, Terraform loses track of all resources it created. Running terraform plan will show every resource as 'to be created' — even if they already exist in AWS.

- Recovery: Use terraform import to re-associate existing resources with the configuration.
- Prevention: Always use remote state (S3) with versioning enabled, so deleted state can be restored from a previous version.

■ *This is why state locking (DynamoDB) and state file backups are critical in production.*

Q. How would you troubleshoot an ImagePullBackOff pod?

- kubectl describe pod — check Events for the exact error (invalid image name, authentication failure, image not found).
- Verify the image name and tag exist in ECR: `aws ecr describe-images --repository-name .`
- Check if the imagePullSecret is correctly configured in the pod spec and the secret exists in the namespace.
- Ensure the node IAM role has the `ecr:GetAuthorizationToken` and `ecr:BatchGetImage` permissions.
- If using a private registry: verify credentials stored in the Kubernetes secret are valid and not expired.

Q. Why should we hire you? (Final answer)

I bring a combination of theoretical knowledge and real, hands-on project experience with the exact tools your team uses — AWS, Kubernetes, Terraform, Jenkins, ArgoCD, and DevSecOps tooling. During my internship, I did not just run pre-built pipelines — I designed, debugged, and improved them.

I am driven by a genuine passion for automation and reliability engineering, and I am at a stage in my career where I am eager to absorb knowledge, contribute meaningfully, and grow quickly. I will be a high-energy, low-friction addition to your team.

Preparation Tip: For a 0–1 year DevOps profile, expect **70% project-based questions**, **20% scenario/troubleshooting**, and **10% HR questions**. The strongest strategy is to explain every tool in your architecture, justify why you chose it, and walk through end-to-end troubleshooting confidently.